

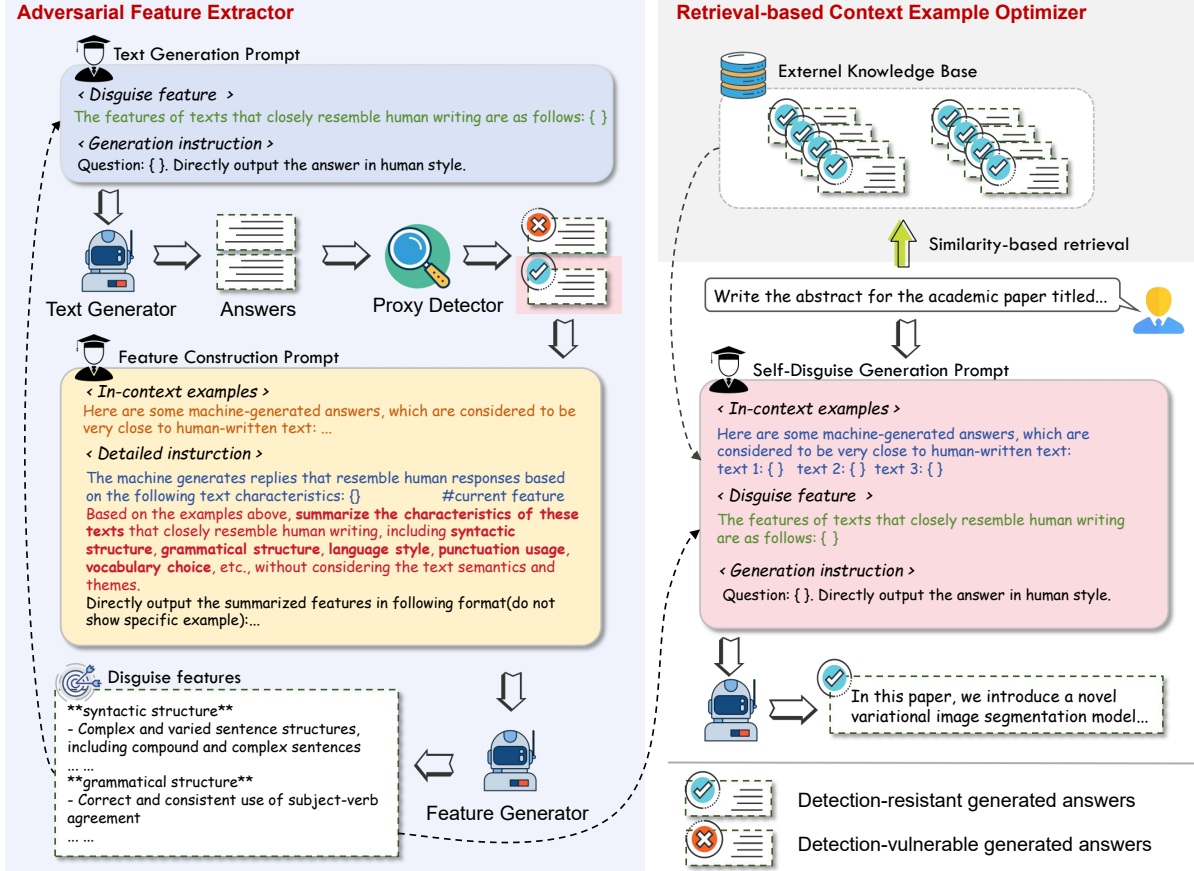


Figure 2: The overall framework of SDA. The SDA consists of an adversarial feature extractor and a retrieval-based context examples optimizer. The former generates disguise features, and the latter identifies the most relevant context examples. In the reference stage, both selected context examples and the extracted disguise features are integrated to guide the LLM in generating detection-resistant text.

Retrieval-based context examples optimizer

Inspired by RAG (Lewis et al. 2020), which enhances the accuracy and richness of LLM by utilizing external knowledge bases, we design the retrieval-based context examples optimizer to further mitigate the limitations of disguise feature prompts on text generation and enhance the self-disguise capability of LLMs. The optimizer comprises two key processes: the construction of an external knowledge base and similarity-based retrieval.

The construction of external knowledge base. For each query $x \in \mathcal{X}$, the corresponding undetectable text y is generated by applying the optimal disguise features obtained through the adversarial process, forming a knowledge pair (x, y) . The set of knowledge pairs $(\mathcal{X}, \mathcal{Y})$ is stored as the external knowledge base. Then, we leverage a pre-trained model f , such as RoBERTa (Liu et al. 2019) to vectorize $\mathcal{X}$, which is defined as:

$$\mathcal{V} = f(\mathcal{X}) \quad (3)$$

We store $\mathcal{V}$ in a vector database, enabling efficient retrieval in subsequent steps.

Similarity-based retrieval. For a given user input x^* , we

first vectorize it into v^* :

$$v^* = f(x^*) \quad (4)$$

Then, we query the vector database to retrieve the top- k closest vectors to v^* using the L2 norm:

$$\mathcal{V}_k = \arg \min_{\substack{\mathcal{S} \subset \mathcal{V} \\ |\mathcal{S}| = k}} \sum_{v_i \in \mathcal{S}} \|v^* - v_i\|_2 \quad (5)$$

where $\mathcal{V}_k$ represents the set of the top- k closest vectors to v^* . The knowledge pairs $(\mathcal{X}_k^*, \mathcal{Y}_k^*)$ corresponding to $\mathcal{V}_k$ are retrieved from the external knowledge, and the set of top- k closest undetectable texts $\mathcal{Y}_k^*$, is used as context examples for the target LLM. The complete process of the algorithm is shown in Appendix C.

Experiments

Experimental Settings

Dataset. We randomly sample 1000 instances from the human-written texts in the RAID dataset (Dugan et al. 2024). The title field of each sample is then extracted and

LLM	Method	Detector				Average	
		RADAR	DeTeCtive	MPU	ChatGPT-Detector		
Qwen-max	Modification-based	Paraphrase	34.50	99.00	36.50	8.00	44.50
		DIPPER	80.00	99.50	50.00	19.50	62.25
		HMGC	47.00	100.00	80.50	0.00	56.88
	Generation-based	SICO	100.00	100.00	66.00	19.00	71.25
		SDA (ours)	34.00	81.00	33.00	21.50	42.39
llama3.3-70B-instruct	Modification-based	Paraphrase	60.00	99.50	62.50	14.50	59.13
		DIPPER	89.00	96.50	61.50	30.00	69.25
		HMGC	60.50	100.00	82.50	0.00	60.75
	Generation-based	SICO	95.00	100.00	61.50	0.00	64.13
		SDA (ours)	62.50	91.50	21.50	11.50	46.75
deepseek-v3	Modification-based	Paraphrase	51.00	99.50	36.50	5.00	48.00
		DIPPER	77.00	98.50	49.00	17.50	60.50
		HMGC	58.00	100.00	38.00	0.00	49.00
	Generation-based	SICO	92.00	99.50	26.50	0.00	54.50
		SDA (ours)	22.00	75.50	6.50	3.00	26.75

Table 1: Accuracy of different evasion methods across three LLMs. The optimal results are highlighted in bold in the table.

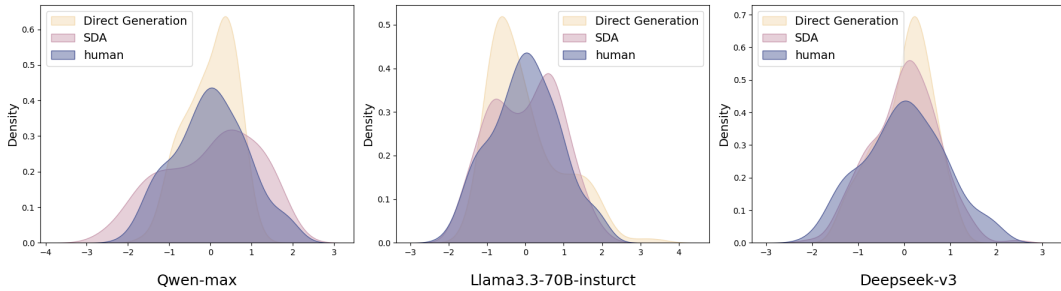


Figure 3: Visualization of Text Distribution Generated by Three LLMs Before and After Using SDA

used to construct the user query set $\mathcal{X}$ in the same manner. For instance, in the case of abstract texts, the corresponding prompt is formulated as: "Write the abstract for the academic paper titled 'title'." The dataset is then split into training, validation, and test sets in a 6:2:2 ratio. Notably, training set here is used for disguise feature extraction and the external knowledge base construction.

AIGT detectors. We employ four AIGT detectors to evaluate the performance of SDA: (1) *ChatGPT-detector* (Guo et al. 2023) is a Roberta-based model fine-tuned on the text generated by GPT-3.5. (2) *Radar* (Hu, Chen, and Ho 2023) is a robust detector based on adversarial learning. (3) *DeTeCtive* (Guo et al. 2024) introduces a multi-task-assisted hierarchical contrastive learning framework to distinguish writing styles of different authors for detecting AIGT. (4) *MPU* (Tian et al. 2024) introduces a multi-scale positive-negative labeling (MPU) training framework, which significantly enhances short text detection performance without compromising long text detection accuracy.

Baselines. We employ four evasion methods, including both modification-based and generation-based techniques,

as baselines to assess the performance of SDA. (1) *paraphrase* rewrites text by GPT-3.5. (2) *DIPPER* (Krishna et al. 2023) rewrites text by fine-tuning T5. (3) *HMGC* (Zhou, He, and Sun 2024) employs adversarial strategies to replace key terms critical for detection with synonyms. (4) *SICO* (Lu et al. 2023) guides the model to generate more human-like text by using samples that have undergone synonym replacement and paraphrasing attacks as in-context examples.

Evaluation Metrics. We employ various evaluation metrics to provide a comprehensive assessment of performance of SDA. (1) *Accuracy*. We evaluate the evasion detection effectiveness of the method by testing the detection accuracy of different detectors on 200 attacked texts. (2) *Cosine Similarity*. We assess the performance of model by evaluating the similarity between the generated text and HWT. (3) *Perplexity (PPL)*. It measures the level of uncertainty of a given document. A lower value is usually associated with higher quality in the generated text. (4) *Self-BLEU*. It is a metric used to measure the diversity of generated text. A lower value indicates higher diversity in the generated text.

LLM Selection. In our experiment, we utilize three latest

LLMs	Qwen-max	llama3.3	deepseek-v3
Direct Generation	21.40	15.33	23.63
Paraphrase	29.80	23.64	30.41
DIPPER	24.62	21.37	25.39
HMGC	22.47	16.24	24.01
SDA (ours)	16.68	15.25	20.47

Table 2: Comparison of the perplexity of texts between SDA and modification-based evasion detection methods.

LLMs, including Qwen-max (Yang et al. 2024), deepseek-v3 (DeepSeek-AI 2024), and LLaMa3.3-70B-instruct (Team and Meta-AI 2024), to validate the effectiveness of SDA. Both the feature generator and the text generator in the adversarial feature extractor utilize Qwen-max. Notably, Different target LLMs are guided by the same disguise features generated by Qwen-max.

Implementation Setting. To simulate the real-world detection scenario, we directly use the publicly available version of the corresponding detector on HuggingFace¹ for our experiments. For all experiments in our study, the proxy detector used is ChatGPT-detector, a RoBERTa-based detector that has been widely applied in previous works (Lu et al. 2023; Dugan et al. 2024). Additionally, we set the detection threshold $\sigma = 0.5$, the optimized step $\eta = 5$, the maximum error range $\delta = 2$ and the number of context examples $k = 5$.

Main Results

We compare our method with four baselines across three different target LLMs. As shown in Table 1, SDA consistently outperforms the other baselines by causing a greater reduction in the average detection accuracy of the four detectors for three target LLMs. For the case of alignment between the object detector and the agent detector (both being ChatGPT-detector), both HMGC and SICO are able to reduce detection accuracy to zero. HMGC is a modification-based attack that iteratively replaces important words in the text until it successfully evades the proxy detector. This ensures that the modified text cannot be detected by the proxy detector, but its performance is not reliable against unknown detectors. SICO, on the other hand, uses manually designed in-context examples to guide the LLM in generating text that avoids detection. We argue that this limits the LLM’s ability to learn effective evasion features, resulting in poor generalization when facing unseen detectors. In contrast, our SDA method learns more general disguise features. It aims to reduce the average detection accuracy across all unseen detectors, even when they differ from the proxy detector. As shown in Table 1, SDA demonstrates strong generalization ability. Additionally, we visualize the text generated using SDA across three LLMs, as shown in Figure 3. Compared to direct generation, the texts generated with SDA are closer to HWT, demonstrating the effectiveness of the proposed SDA.

¹<https://huggingface.co/>

LLMs	metric	SICO	SDA(ours)
Qwen-max	PPL ↓	16.55	16.68
	cosine similarity ↑	0.9877	0.9946
	self-BLEU ↓	0.5510	0.4920
llama3.3	PPL ↓	15.30	15.25
	cosine similarity ↑	0.9816	0.9944
	self-BLEU ↓	0.5454	0.4952
deepseek	PPL ↓	19.64	20.47
	cosine similarity ↑	0.9913	0.9977
	self-BLEU ↓	0.5084	0.4383

Table 3: Comparison of the text quality between SDA and generation-based evasion detection methods.

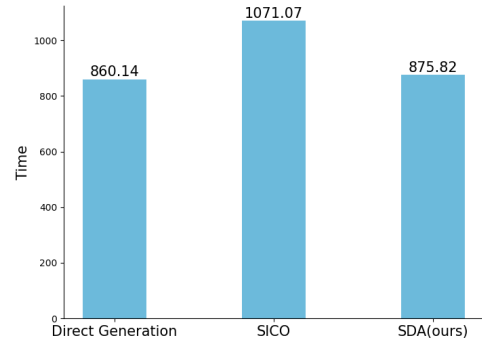


Figure 4: Comparison of generation time.

Evaluation of Text Quality

Comparison with Modification-based Methods. Since diversity metrics primarily evaluate the generation capabilities of language models, we focus solely on the perplexity of the attacked text for the modification-based methods. As shown in Table 2, modification-based approaches tend to degrade text generation quality, whereas our proposed SDA effectively enhances the quality of AIGT. Specifically, on Qwen-Max, our approach achieves a perplexity reduction of 4.72 relative to direct generation.

Comparison with Generation-based Methods. We evaluate the cosine similarity between the generated text and HWT under different methods, while using self-BLEU to measure the diversity of AIGT. The results are shown in Table 3. However, compared to SICO, the cosine similarity of SDA is higher across all three LLMs, indicating that SDA can effectively mimic HWT. Furthermore, the diversity of text generated using SDA surpasses that of SICO across all three LLMs, suggesting that SDA effectively mitigates the influence of prompts on AIGT diversity.

We further conduct human evaluation for these methods across three dimensions: fluency, semantic clarity, and the probability of AI generation, as detailed in Appendix D.